PHP has a wide variety of functions and modules for hashing. We'll have a quick look here at the following core functions.

Table 8-21. *Hash/error check functions*

`crc32()`	Calculate the crc32 value of a string (used for error checking)
`md5()`	Calculate md5 hash value of a string
`sha1()`	Calculate sha1 hash value of a string

crc32()

Cyclic redundancy check (or checksum) is a space-efficient method of generating a value that can be used to verify binary data. The output is an integer, and two identical strings always generate the same value. It is, of course, possible for two different strings to give the same value, because the number of possible outputs of `crc32()` is limited to the range of possible integer values for the system on which it runs, which is finite (see Chapter 7, "Numbers," for more information on integer limits). This function is best deployed only for error checking (which is the main application of the algorithm – verifying network data packets).

```
print crc32('b5a7b602ab754d7ab30fb42c4fb28d82') . PHP_EOL;
print crc32('d19f2e9e82d14b96be4fa12b8a27ee9f');
```

```
Output:
2575120314
2575120314
```

The code above demonstrates a `crc32()` collision.

md5(), md5_file()

An alternative to integer-based `crc32` is `md5()`. Based on the MD5 Message-Digest Algorithm, `md5()` is a fast hashing function that produces a wider range of potential (fixed width) string values than the integers of `crc32()`, so the collision probability is a lot lower (128 bits of output, 2^{128} possible values, small but still non-zero). Collisions are a problem because it is possible for two different values to appear to be identical, files such as SSL certificates, for example. `md5()` accepts a string argument and outputs the